

Academic Year 2025-2026, Week ≥ 9

*Practical work proposal – to be developed by each group of 2 or 3 students
(report to be submitted by each group of 2 or 3 students on Moodle by May 31st)*

Challenge:

We want to develop a (Matlab, Python) software application that recognizes a song/music using just a 10-second sound clip of it. This system will record/read a 10-second segment of a song/music to be identified, will extract meaningful features from it so as characterize the fingerprint (sound signature, or “DNA”) of that song/music and, using this fingerprint, will search in a database for the best match. Prototyping will start by designing an offline functional modality. Ideally, time and resources permitting, the objective at the end of the semester is to showcase a real time functional modality. In the following, we describe the basic system architecture and essential signal processing steps.

Main system parameters and functionality steps:

Each group of students will build a database containing between 20 and 30 songs/musics to be freely selected by that group of students [**this has been done for you already, see below**]. Each song/music will contain just a 30-second audio segment that is representative of that given song (that segment may not contain audio interruptions or long silence regions). The sampling frequency of all audio recordings should be 22050 Hz.

For each song/music, a fingerprint will be extracted based on relevant audio features, e.g., simple non-linear frequency descriptors such as the average MFCC (Mel-frequency Cepstral Coefficients) vector. A fingerprint database will be constructed that will facilitate search and identification operations.

The system must be able to read or to record a 5-10 second audio clip of a song/music to be identified. After that, the system should be able to extract a fingerprint from that short audio clip and search the fingerprint database for the best match. It should then play the audio clip as well as the corresponding best match song/music.

Essential processing workflow:

1. Database pre-processing

- a. [**this has been done already for you, the database is available on Moodle**] create a folder storing all 20-30 selected songs, each song being represented by a representative segment of that song lasting 30 seconds (including a smooth fade-in region at the start of the representative segment lasting around 1 second, and a smooth fade-out region at the end of the representative segment lasting around 1 second).
- b. for each one of the selected songs, extract the corresponding fingerprint using the processing steps indicated next. A fingerprint is obtained as the average (or mean) of all MFCC vectors (MFCC= Mel-Frequency Cepstral Coefficients) of each song. Figure 1 illustrates the main steps involved in the computation of each MFCC vector. In the following, each step is briefly described. [**these steps are all implicitly implemented in the Matlab command “melSpectrogram” whose usage is illustrated in the Matlab command file available on Moodle**]

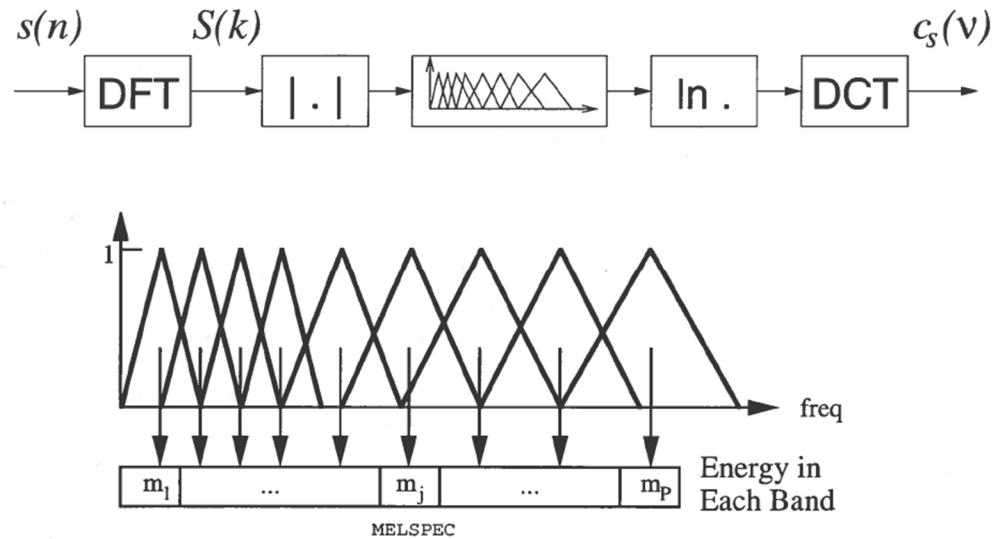


Figure 1: Compact block diagram of the main processing steps allowing to compute MFCC vectors.

- i. Reading/recording: an audio signal is either read from an audio file, or is recorded from an audio input (microphone or audio Line In) using an audio board.
- ii. Framing: The audio signal is segmented into small, overlapping frames (each lasting around 40 ms). The reason for this is that the audio signal changes over time, and a single signal analysis involving the total duration of the audio signal (lasting several seconds) would lose time and frequency details. In Fig. 1, a frame of audio samples is represented by $s(n)$.
- iii. Windowing: A window function (e.g., a Sine, or a Hamming window) is applied to each frame to reduce signal discontinuities at the edges. A 50% overlap between adjacent windows will be considered in order to ensure a more detailed signal analysis. The 50% window overlap is illustrated in Fig. 2.

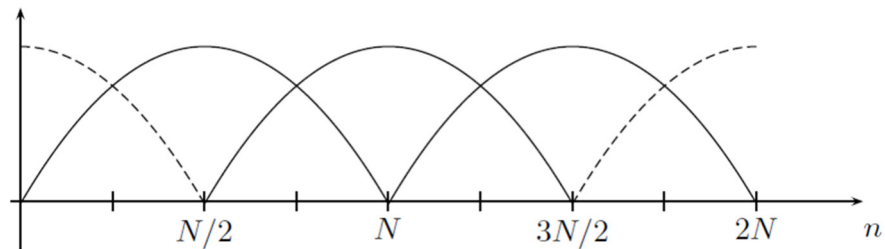


Figure 2: Illustration of the 50% overlap between adjacent segmentation windows. N represents the number of audio samples in each window.

- iv. Discrete Fourier Transform (DFT, computed by means of a fast algorithm called Fast Fourier Transform -FFT): Converts each frame from the time domain to the frequency domain (spectrum) to estimate the periodogram (i.e., the power spectrum).
- v. Mel Filterbank: The power spectrum is passed through a set of triangular band-pass filters (between 13 and 256 filters; use 128 as a start) mapped to the Mel scale, which mimics the human perception by focusing on lower frequencies and providing less spectral resolution at higher frequencies.
- vi. Logarithm: The (natural) logarithm of the filterbank energies is computed to simulate the human ear's non-linear response to sound intensity.

- vii. Discrete Cosine Transform (DCT): Finally, the DCT is applied to the log filterbank energies to decorrelate the filterbank coefficients, producing the MFCC vector representing the audio frame being processed.
- viii. The above steps are repeated for all the frames in the audio signal under analysis. Finally, the average MFCC vector is computed and that represents our fingerprint of the audio signal (song, or clip). [NOTE: the Matlab command “melSpectrogram” delivers all MFCC vectors identified in the audio signal under analysis, students should compute the mean MFCC vector which is taken as the “DNA” identifier of the audio signal they are extracted from]

2. Best match identification.

After the fingerprints of the reference songs are pre-computed, the system is ready to perform the processing steps leading to identification of a short audio clip. [NOTE: the audio clips are also available on Moodle, their duration is 10 seconds and they belong to a song region different from the 30-second song reference] The following steps should be implemented to identify an audio clip:

- a. an audio clip is read and its fingerprint is extracted; the processing procedure is the same as that used to extract the fingerprints of the reference songs;
- b. the clip fingerprint is compared to all pre-computed fingerprints; the best match signals the best identification guess by the system; this process is implemented as described next:
 - i. if $MFCC_{song}$ represents the fingerprint of a song, and $MFCC_{clip}$ represents the fingerprint of the audio clip, a comparison can be made by computing the cosine similarity, also known as cross-correlation, or Pearson coefficient, as follows

$$CS = \frac{\sum_k (MFCC_{song}[k] \cdot MFCC_{clip}[k])}{\sqrt{\sum_k |MFCC_{song}[k]|^2 \cdot \left(\sum_k |MFCC_{clip}[k]|^2\right)}}$$

NOTE: in Matlab, if A and B are two vectors, their cross correlation can be computed using $CS = \text{corrcoef}(A, B)$

- ii. after all similarities are computed, the largest similarity is found and is used to declare the best match, i.e. the best correspondence between the audio clip and all the songs in the database.
- c. after the best match is identified, both the audio clip and the identified song should be played for human verification.

3. Bonus appreciation

A bonus project evaluation will be made if one or several of the following possibilities are exploited / demonstrated:

- a. a match confidence level (let us say, between 0% and 100%) is indicated to suggest the confidence level of that established clip-song match (a number below 50% means low confidence level); this confidence level may be computed using a basic criterion;
- b. an overall success rate is provided that characterizes the performance of the recognition system (the clip database contains 22 clips, what is the percentage of the correctly identified clips ?);
- c. attempts to improve the baseline recognition performance, for example, by modifying some parameters in the recognition system (such as the length of the MFCCC vectors);
- d. a real-time functionality is demonstrated whereby the system “listens” to an audio clip (e.g., using a microphone), after which the song identification processing is carried out and the best match is indicated by the system.